

Quick Start Cookbooks

Immediate-value workflows for Codex and Claude Code

OUTCOMES

- | | |
|----|----------------------------------|
| 01 | Connect Codex or Claude Code |
| 02 | PR component and CI readiness |
| 03 | Daily deployment digest |
| 04 | Failed CI build triage |
| 05 | nCino build health |
| 06 | Audit activity review |
| 07 | Controlled build or quick deploy |

Outcome-oriented workflows for PR validation, ARM CI checks, deployment reporting, nCino build health, and audit review. Built from the verified ARM MCP server contract.

Edition: 2026-08-11

ARM MCP Quick Start Cookbooks

Use AutoRABIT ARM from Codex or Claude Code to answer a release question in minutes, then expand the workflow only when the first result is useful.

Start here

1. [Connect Codex or Claude Code](#)
2. Pick one outcome:
 - [PR component and CI readiness review](#)
 - [Daily deployment digest](#)
 - [Failed CI build triage](#)
 - [nCino build health report](#)
 - [Audit activity review](#)
 - [Controlled build or quick deploy](#)

An AI client needs three things to create value from ARM:

1. A trusted ARM endpoint and token.
2. A narrow operational question.
3. A tool result that can prove the answer.

MCP supplies the tool contract. Codex or Claude Code supplies planning, synthesis, and artifact creation. ARM remains the system of record for ARM status.

The hard part is evidence alignment. A Git diff, CI build, deployment iteration, Jira story, and audit event can describe different slices of the same release. A polished summary can still be wrong when those identifiers do not line up.

Every cookbook therefore makes identifiers explicit, keeps independent status fields separate, and labels unavailable evidence as unavailable.

Use a repeatable loop:

1. Resolve an exact job, deployment label, build number, iteration, or date range.
2. Read ARM data through dedicated MCP tools.
3. Reconcile status and traceability across returned records.
4. Produce a reviewable artifact with evidence and gaps.
5. Request explicit approval before any mutation.

Connect the server, paste one cookbook prompt, replace the angle-bracket placeholders, and run it.

The current server exposes 27 dedicated tools:

- 11 standard CI job tools
- 7 nCino CI job tools
- 6 deployment reporting tools
- 3 SIEM audit tools

It also exposes six reference resources and eight guided prompts at the MCP protocol layer. Client presentation varies. The natural-language prompts in these cookbooks work in both Codex and Claude Code.

The optional generic `arm_call_api` tool is disabled by default. None of these quick starts require it.

Value ladder

Level	Outcome	ARM MCP alone	Extra integration
1	Answer one release question	Yes	None
2	Produce a repeatable report	Yes	None
3	Enrich with live PR, Jira, or Confluence data	Partial	GitHub, Jira, or Confluence MCP/API
4	Post comments or publish reports	No	Write-enabled destination integration
5	Trigger a build or deployment	Yes	Explicit operator approval

Evidence rules

- Preserve exact case-sensitive job names, labels, build numbers, and iteration numbers.
- Keep build, deployment, post-deployment, quick-deploy, rollback, and coverage statuses separate.
- Use `UNKNOWN` when required data is missing or inaccessible.
- Cite the MCP tool used for every decision row.
- Do not call `arm_trigger_build`, `arm_quick_deploy`, `arm_start_rollback`, `arm_abort_ci_job`, `arm_update_baseline_revision`, or `arm_ncino_trigger_build` unless the prompt explicitly authorizes the mutation.
- Treat a tool transport or authentication failure as an access problem, never as an empty result.

Connect ARM MCP to Codex or Claude Code

Outcome: a verified, local ARM MCP connection without placing an ARM token in version control.

Time: about 10 minutes after you have an ARM API token.

1. Build the server

Requirements:

- Node.js 22 or newer
- An AutoRABIT ARM tenant URL
- An ARM API token

From the server checkout:

```
cd "/absolute/path/to/ARM-MCP-Server"
npm ci
npm run build
```

Set credentials in the shell that will launch Codex or Claude Code:

```
export ARM_MCP_SERVER_PATH="/absolute/path/to/ARM-MCP-Server"
export ARM_BASE_URL="https://your-tenant.autorabit.com"
export ARM_API_TOKEN="replace-with-your-token"
```

Use the tenant origin only. Do not append an ARM API path.

Current distribution note, verified 2026-08-11: @autorabit-ai/arm-mcp-server is not available from the public npm registry. Use the source-build path above. Replace this note with the `npm` path after publication is verified.

2A. Connect Codex

Codex reads global MCP configuration from `~/ .codex/config.toml` and project configuration from `.codex/config.toml` in a trusted project. Add this block to either file:

```
[mcp_servers.autorabit-arm]
command = "node"
args = ["/absolute/path/to/ARM-MCP-Server/dist/index.js"]
env_vars = ["ARM_BASE_URL", "ARM_API_TOKEN"]
required = true
default_tools_approval_mode = "writes"
startup_timeout_sec = 20
tool_timeout_sec = 120
```

The `env_vars` setting forwards the two variables from the shell environment. It keeps their values out of the TOML file.

Verify:

```
codex mcp list
codex
```

Inside Codex, run `/mcp` and confirm `autorabit-arm` is active.

If you prefer the UI, open MCP server settings in the ChatGPT desktop app or Codex IDE extension, add a STDIO server with command `node` and the absolute `dist/index.js` argument, save, and restart that client.

2B. Connect Claude Code

Create `.mcp.json` at the root of the project where you will use Claude Code:

```
{
```

```

    "mcpServers": {
      "autorabit-arm": {
        "type": "stdio",
        "command": "node",
        "args": ["${ARM_MCP_SERVER_PATH}/dist/index.js"],
        "env": {
          "ARM_BASE_URL": "${ARM_BASE_URL}",
          "ARM_API_TOKEN": "${ARM_API_TOKEN}"
        }
      }
    }
  }
}

```

The placeholders are environment-variable references. Do not replace them with secrets in a file that will be committed.

Verify:

```

claude mcp list
claude

```

The first interactive launch may ask you to trust the workspace and approve the project-scoped server. Inside Claude Code, run `/mcp` and confirm the server is connected.

Claude Code can expose ARM MCP prompts as commands and ARM resources through its resource picker. Type `/` or `@` and select the dynamically discovered entry. The full natural-language prompts in this cookbook remain the portable path across both clients.

3. Run a read-only smoke test

Paste this prompt into Codex or Claude Code:

```

Use only the `autorabit-arm` MCP server.

1. Call `arm_list_ci_jobs`.
2. Return the first five exact, case-sensitive CI job names in a table.
3. State the MCP tool used and whether the call succeeded.
4. If authentication, transport, or server startup fails, report the exact error and stop.

Do not call any mutation tool.

```

Success means the client returns real CI job data from the target tenant. A connected status proves process startup; the smoke test proves an authenticated ARM request.

4. Optional audit configuration

The audit tools use a separate host and bearer token. Add these variables only when you need the audit cookbook:

```

export ARM_AUDIT_BASE_URL="https://your-audit-host.example.com"
export ARM_AUDIT_API_TOKEN="replace-with-your-audit-token"

```

For Codex, extend `env_vars`:

```

env_vars = [
  "ARM_BASE_URL",
  "ARM_API_TOKEN",
  "ARM_AUDIT_BASE_URL",
  "ARM_AUDIT_API_TOKEN"
]

```

For Claude Code, extend the server `env` object:

```

{
  "ARM_AUDIT_BASE_URL": "${ARM_AUDIT_BASE_URL}",
  "ARM_AUDIT_API_TOKEN": "${ARM_AUDIT_API_TOKEN}"
}

```

Fast diagnosis

Symptom	Check
Server fails to start	<code>node --version</code> is 22+, and <code>dist/index.js</code> exists
Connected, ARM call fails	<code>ARM_BASE_URL</code> and <code>ARM_API_TOKEN</code> exist in the client process environment
HTTPS validation fails	Tenant URL uses HTTPS and contains no API path
Tool times out	Raise client tool timeout above the server request timeout
Claude project server is pending	Launch <code>claude</code> interactively and approve the workspace/server
Audit tools fail while ARM tools work	Configure the separate <code>ARM_AUDIT_*</code> variables

Sources

- [ARM MCP requirements and configuration](#)
- [OpenAI Codex MCP configuration](#)
- [Claude Code MCP configuration](#)

Quick Start: PR Component and CI Readiness

Outcome: a read-only `READY`, `NOT READY`, or `UNKNOWN` recommendation backed by the local change set and ARM CI evidence.

Time: 5-10 minutes.

What this recipe uses

- Local Git or a connected GitHub tool for the PR change set
- `arm_list_ci_jobs`
- `arm_latest_results`
- `arm_poll_job_status` when the build is still running
- Optional deployment tools when a deployment label and iteration are available

The ARM MCP server does not expose GitHub comment tools or dedicated CodeScan tools. Connect those systems separately if you want automated comment posting or CodeScan-specific evidence.

Inputs

Replace:

- `<BASE_REF>` - target branch or commit, such as `origin/main`
- `<HEAD_REF>` - PR branch or commit, such as `HEAD`
- `<CI_JOB_NAME>` - exact, case-sensitive ARM CI job name
- `<BUILD_NUMBER_OR_LATEST>` - a positive build number or `latest`
- `<MIN_COVERAGE>` - your gate, such as `75`
- `<DEPLOYMENT_LABEL_OR_NONE>` - exact ARM deployment label or `none`
- `<ITERATION_OR_LATEST>` - positive iteration or `latest`

Paste this prompt

Run a read-only PR component and ARM CI readiness review.

Inputs:

- base_ref: <BASE_REF>
- head_ref: <HEAD_REF>
- ci_job_name: <CI_JOB_NAME>
- build_number: <BUILD_NUMBER_OR_LATEST>
- minimum_coverage_percent: <MIN_COVERAGE>
- deployment_label: <DEPLOYMENT_LABEL_OR_NONE>
- deployment_iteration: <ITERATION_OR_LATEST>

Rules:

- Do not call any mutation tool.
- Preserve exact names, paths, build numbers, labels, and statuses.
- Mark missing or inaccessible evidence as UNKNOWN.
- Never infer success from an empty response.

Step 1 - component audit:

1. Compare base_ref with head_ref using the available local Git or GitHub read tools.
2. List every added, modified, renamed, and deleted path.
3. For Salesforce metadata paths, derive component type and component name only when the path proves them. Label everything else `unclassified`.
4. Detect duplicate, generated, unrelated, or unexpected files and explain the exact rule used.
5. Return `CLEAR`, `MISMATCH`, or `UNKNOWN` for the component audit.

Step 2 - ARM CI and validation:

1. Call `arm\_list\_ci\_jobs` and require one exact, case-sensitive match for ci_job_name.
2. Call `arm\_latest\_results` for ci_job_name.
3. If a specific build number was supplied or the latest build is not terminal, call `arm\_poll\_job\_status` with the selected build number.
4. Extract each ARM status field separately. Extract test, coverage, and scan evidence only when the response contains it.
5. If deployment_label is not `none`, call `arm\_get\_deployment`, resolve the requested or latest iteration, then call `arm\_get\_deployment\_components` and `arm\_get\_deployment\_test\_coverage`.
6. Compare ARM deployment components with the PR component inventory when both are available. Report exact extras and omissions.

Decision:

- READY: component audit is CLEAR, selected ARM build is terminal and successful, every required validation returned success, and numeric coverage is at least minimum_coverage_percent.
- NOT READY: any proven mismatch, failed ARM status, failed validation, or numeric coverage below the threshold.
- UNKNOWN: any required evidence is missing, inaccessible, stale, ambiguous, or refers to a different build or iteration.

Output exactly these sections:

1. Verdict - one line with READY, NOT READY, or UNKNOWN
2. Identity - base/head refs, CI job, build number, deployment label, iteration
3. Component audit - counts by action and component type, then exact mismatches
4. ARM checks - one row per returned status or validation field with value and source tool
5. Coverage - observed value, threshold, and source; use unavailable when absent
6. Blocking findings - evidence-backed only
7. Missing evidence
8. Suggested PR comment - concise, factual, and comment-only; do not post it

Expected decision boundary

The AI recommendation remains advisory. ARM's returned status supplies the ARM gate evidence. A separate GitHub integration and explicit authorization are required to post the comment or change merge state.

Fast expansion

After the read-only result is reliable:

1. Connect a GitHub MCP server.
2. Add CodeScan results through a verified API or MCP contract.

3. Post the generated comment only after the evidence identifiers match the PR head SHA and selected ARM build.
4. Keep the merge action outside the agent until the team defines an explicit authorization policy.

Source

- [ARM CI and deployment tool catalog](#)

Quick Start: Daily Deployment Digest

Outcome: a reviewable Markdown and CSV deployment report for one date range and destination org.

Time: 10-15 minutes.

What this recipe uses

- arm_list_deployments
- arm_get_deployment
- arm_get_deployment_components
- arm_get_deployment_stories
- arm_get_deployment_promotion_log
- arm_get_deployment_test_coverage

ARM already supplies deployment, component, Jira-linked commit, log, and coverage evidence. Live PR metadata, Jira ownership, Excel formatting, and Confluence publication require additional tools only when ARM does not return those fields.

Inputs

Replace:

- <FROM_DATE> and <TO_DATE> - YYYY-MM-DD
- <DESTINATION_ORG> - **exact destination Salesforce org**, or `all`
- <LIMIT> - `1-100`
- <OUTPUT_DIR> - **such as** `reports/arm/2026-08-11`

Paste this prompt

Create a read-only daily ARM deployment digest.

Inputs:

- from_date: <FROM_DATE>
- to_date: <TO_DATE>
- destination_org: <DESTINATION_ORG>
- limit: <LIMIT>
- output_dir: <OUTPUT_DIR>

Rules:

- Use only read-only ARM MCP tools and local file creation.
- Do not publish, upload, comment, trigger, deploy, roll back, abort, or update a baseline.
- Preserve every exact deployment label, status, org, iteration, story ID, commit ID, user, and timestamp returned by ARM.
- Mark unavailable fields as `unavailable`; do not infer them.

Steps:

1. Call `arm\_list\_deployments` with fromDate, toDate, limit, and destSfOrg when destination_org is not `all`.
2. If the authenticated call succeeds with zero deployments, report zero. If the call fails, report unavailable and stop.
3. For each returned deployment label:
 - a. Call `arm\_get\_deployment`.
 - b. Resolve `latestIterationNumber` from detail.
 - c. Call `arm\_get\_deployment\_components`.
 - d. Call `arm\_get\_deployment\_stories` for that latest iteration when supported by the returned data.
 - e. Call `arm\_get\_deployment\_promotion\_log` for the latest iteration.
 - f. Call `arm\_get\_deployment\_test\_coverage` for the latest iteration.
4. Reconcile each result by exact deployment label and iteration. Do not combine data across labels or iterations.
5. Create output_dir if needed.
6. Write `deployment-digest.md` with:
 - executive summary
 - counts by exact ARM status
 - counts by destination org
 - one deployment evidence table
 - failed or non-terminal deployments
 - coverage failures or unavailable coverage
 - notable promotion-log diagnostics
 - missing evidence
7. Write `deployment-components.csv` using Python's standard `csv` module. Use one row per returned component with these columns:
deployment_label, iteration, deployment_status, source_environment, destination_environment, component_type, component_name, file_path, change_type, jira_story, commit_id, triggering_user, coverage_status, evidence_tools
8. In every summary row, list the ARM MCP tools that supplied the evidence.
9. Return the two file paths, the deployment count, and any incomplete records.

What good looks like

- Every record maps to one exact label and iteration.
- Status counts use ARM's returned values without normalization that hides differences.
- The report distinguishes zero deployments from failed access.
- The output can be regenerated for the same date range.

Expand the workflow

Add integrations one evidence gap at a time:

Need	Add	Result
Live PR number, author, head SHA	GitHub MCP/API	PR traceability

Need	Add	Result
Jira assignee and requester	Jira MCP/API	Ownership enrichment
Monthly workbook, one tab per day	Python with <code>openpyxl</code>	Excel artifact
Scheduled publication	Confluence MCP/API plus scheduler	8 AM/7 PM delivery

Before publishing, compare record counts and a sample of exact labels/components between the generated artifact and ARM tool responses.

Source

- [ARM deployment reporting tools](#)

Quick Start: Failed CI Build Triage

Outcome: an evidence-backed failure summary and next action without changing ARM state.

Time: 5 minutes.

What this recipe uses

- arm_list_ci_jobs
- arm_latest_results
- arm_poll_job_status
- arm_ci_job_history
- arm_rollback_history
- arm_rollback_details

Inputs

Replace <CI_JOB_NAME> and <BUILD_NUMBER_OR_LATEST>.

Paste this prompt

Investigate a failed or stalled ARM CI build. Keep the workflow read-only.

Inputs:

- ci_job_name: <CI_JOB_NAME>
- build_number: <BUILD_NUMBER_OR_LATEST>

Rules:

- Do not call trigger, quick deploy, rollback, abort, or baseline-update tools.
- Require an exact, case-sensitive CI job match.
- Keep build status, quick-deploy status, rollback validation, tests, coverage, scans, and deployment status separate.
- Label interpretation as `proven`, `likely`, or `unknown`.

Steps:

1. Call `arm\_list\_ci\_jobs` and verify one exact match for ci_job_name.
2. Call `arm\_latest\_results` for the job.
3. Resolve the selected build number. If the user supplied a number, do not silently replace it with latest.
4. Call `arm\_poll\_job\_status` for the selected build.
5. Call `arm\_ci\_job\_history` and use the smallest range that includes recent comparable builds.
6. If ARM returns rollback-related evidence, call `arm\_rollback\_history` and `arm\_rollback\_details`. These are read-only checks.
7. Compare the selected build with recent successful and failed builds only when exact comparable fields are available.

Output:

1. One-line status: FAILED, STALLED, IN PROGRESS, SUCCESSFUL, or UNKNOWN
2. Exact identity: job and build number
3. Timeline: returned stages and timestamps in order
4. Failure evidence: exact returned messages, failed stages, tests, components, or validations
5. Regression clues: differences from recent comparable builds
6. Rollback evidence: eligibility/validation/history as separate fields
7. Root-cause assessment:
 - proven: directly returned by ARM
 - likely: supported by at least two observations
 - unknown: missing evidence
8. Next action: one smallest reversible diagnostic step
9. Operator options: draft-only commands or ARM actions; do not execute them

For every factual row, include the source MCP tool.

Follow-up prompt for one failure

Take the highest-confidence failure from the previous report.

Trace it to the narrowest returned stage, component, test, log message, or status transition. Quote only the minimum exact error fragment needed. Separate ARM evidence from your inference. Recommend one diagnostic that can falsify the leading hypothesis. Do not mutate ARM state.

Source

- ARM CI Jobs v1 tools

Quick Start: nCino Build Health

Outcome: a read-only nCino build report that keeps feature, deployment, post-deployment, and rollback outcomes separate.

Time: 5-10 minutes.

What this recipe uses

```
- arm_ncino_list_ci_jobs
- arm_ncino_get_latest_build
- arm_ncino_get_build_summary
- arm_ncino_poll_build_status
- arm_ncino_get_build_history
```

Inputs

Replace <NCINO_JOB_NAME> and <BUILD_NUMBER_OR_LATEST>.

Paste this prompt

Generate a read-only nCino CI build health report.

Inputs:

```
- job_name: <NCINO_JOB_NAME>
- build_number: <BUILD_NUMBER_OR_LATEST>
```

Rules:

- Do not call `arm\_ncino\_trigger\_build` or any other mutation tool.
- Require one exact, case-sensitive match for job_name.
- If build_number is supplied, use that build. If it is `latest`, resolve the number from ARM.
- Keep every ARM status field separate when fields disagree.
- State missing fields as unavailable. Do not infer success from an empty field.

Steps:

1. Call `arm\_ncino\_list\_ci\_jobs` and verify the exact job.
2. Call `arm\_ncino\_get\_latest\_build`; use its buildNumber only when the input is `latest`.
3. Call `arm\_ncino\_get\_build\_summary` for recent context.
4. Call `arm\_ncino\_poll\_build\_status` for the selected job and build number.
5. Call `arm\_ncino\_get\_build\_history` for feature and version-level outcomes.

Output:

1. Overall classification: HEALTHY, FAILED, IN PROGRESS, DEGRADED, or UNKNOWN
2. Identity: exact job, build number, title, destination org, trigger identity, timestamps
3. Status matrix: build, deployment, post-deployment, rollback, metadata retrieval, data retrieval, and any other returned status as separate rows
4. Failed features or versions: exact names, versions, stages, and returned messages
5. Rollback: enabled, eligible, validated, attempted, and result as separate returned fields
6. Recent context: comparable builds only; explain the comparison key
7. Missing or contradictory evidence
8. One recommended next diagnostic action

Include the source MCP tool for every evidence row.

Claude Code shortcut

The server exposes `arm_ncino_build_report_guide` as an MCP prompt. In Claude Code, type `/`, select the dynamically discovered prompt for the `autorabit-arm` server, and supply the job name and optional build number. Use the full prompt above when you need the strict output contract.

Source

- ARM nCino CI Jobs v1 tools

Quick Start: Audit Activity Review

Outcome: a bounded, read-only summary of ARM audit activity from CEF-formatted SIEM logs.

Time: 5-10 minutes.

Prerequisite: configure the separate `ARM_AUDIT_BASE_URL` and `ARM_AUDIT_API_TOKEN` values in [client setup](#).

What this recipe uses

- `arm_audit_list_event_types`
- `arm_audit_get_logs`
- Optional `arm_audit_download_logs` for a bounded ZIP export

Inputs

Replace:

- `<START_TIME_ISO>` - such as `2026-08-11T00:00:00Z`
- `<EVENT_TYPES>` - comma-separated values such as `LOGIN, DEPLOYMENT, CIBUILD`, or `all`
- `<MAX_RESULTS>` - positive integer

Paste this prompt

Review AutoRABIT ARM audit activity. Keep the workflow read-only.

Inputs:

- `start_time: <START_TIME_ISO>`
- `event_types: <EVENT_TYPES>`
- `max_results: <MAX_RESULTS>`

Rules:

- Use the separate ARM audit MCP tools.
- Do not treat an authentication, transport, timeout, or parse failure as zero events.
- Preserve exact timestamps, event types, severities, users, and object identifiers.
- Keep observed facts separate from anomaly hypotheses.

Steps:

1. Call ``arm_audit_list_event_types``.
2. Validate each requested event type against the returned list. If `event_types` is ``all``, omit the filter.
3. Call ``arm_audit_get_logs`` with `startTime`, `maxResults`, and the validated comma-separated `eventType` filter.
4. Parse each CEF entry into `timestamp`, `vendor`, `product`, `productVersion`, `eventType`, `name`, `severity`, and `extensions`.
5. Count only successfully parsed entries. Report unparsed entries separately.

Output:

1. Query result: succeeded, failed, or partial
2. Scope: exact start time, filters, max results, returned count, parsed count
3. Counts by exact event type and severity
4. Activity timeline in chronological order
5. Identity summary: users or actors exactly as returned
6. Notable sequences: failed logins, CI builds, deployments, commits, or merges when observed
7. Anomaly hypotheses: evidence, competing explanation, and confidence
8. Data-quality gaps and truncation risk
9. Two narrower follow-up queries

Include ``arm_audit_get_logs`` as the evidence source for observed events.

Optional evidence export

The server limits audit ZIP ranges to 90 days and writes the file to `ARM_AUDIT_DOWNLOAD_DIR` or the operating-system temp directory.

Download an ARM audit evidence ZIP for this exact bounded range:

- `start_time`: <START_TIME_ISO>
- `end_time`: <END_TIME_ISO>

Call ``arm_audit_download_logs`` once. Return the resource-link path, byte size when provided, and requested range. Do not open, move, upload, or delete the ZIP.

Source

- ARM SIEM audit tools

Quick Start: Controlled Build or Quick Deploy

Outcome: prepare, approve, execute, and verify one ARM mutation with exact identifiers.

Time: 5-15 minutes, depending on build duration.

This cookbook has two phases. Phase A reads and prepares. Phase B runs only after an operator provides the exact approval phrase.

Option 1: Trigger and monitor a CI build

Replace <CI_JOB_NAME>, <PROJECT_NAME>, and <BUILD_TITLE>.

Phase A - prepare

Prepare an ARM CI build. Do not trigger it yet.

Inputs:

- ci_job_name: <CI_JOB_NAME>
- project_name: <PROJECT_NAME>
- title: <BUILD_TITLE>

1. Call `arm\_list\_ci\_jobs` and require one exact, case-sensitive match for ci_job_name.
2. Call `arm\_latest\_results` and `arm\_poll\_job\_status` for the latest build.
3. Report any current in-progress build or ambiguous project/job mapping.
4. Show the exact proposed MCP call:
tool: arm_trigger_build
arguments: projectName and title
5. State that mutation tools do not retry automatically.
6. Stop. Do not call `arm\_trigger\_build`.

The only authorization phrase for Phase B is:

APPROVE ARM BUILD: <PROJECT_NAME> | <BUILD_TITLE>

Phase B - execute after exact approval

The operator supplied the exact authorization phrase from Phase A.

1. Verify that projectName and title exactly match the approved values.
2. Call `arm\_trigger\_build` once.
3. Capture the returned build number (`cyclenum`) when present.
4. Call `arm\_poll\_job\_status` for the exact ciJobName and returned build number.
5. Poll at most six total times. If still non-terminal, stop and report IN PROGRESS.
6. Report trigger response, build number, current status, quick-deploy status, rollback validation flag, and every MCP tool called.
7. Do not quick deploy, abort, roll back, or update a baseline.

Option 2: Quick deploy a validated build

Replace <CI_JOB_NAME>, <BUILD_NUMBER>, <PROJECT_NAME>, and <BUILD_TITLE>.

Phase A - validate and prepare

Prepare a quick deploy. Do not deploy yet.

Inputs:

- ci_job_name: <CI_JOB_NAME>
- build_number: <BUILD_NUMBER>
- project_name: <PROJECT_NAME>
- title: <BUILD_TITLE>

1. Call `arm\_list\_ci\_jobs` and require one exact, case-sensitive job match.
2. Call `arm\_poll\_job\_status` for the exact build number.
3. Call `arm\_latest\_results` and reconcile its build identity with the requested build.
4. Report build status, quick-deploy status, rollback validation, tests, scans, and coverage exactly as returned.
5. If the requested build cannot be proven eligible from returned evidence, classify BLOCKED.
6. Show the exact proposed MCP call:
tool: arm_quick_deploy
arguments: ciJobName, buildNumber, projectName, title
7. Stop. Do not call `arm\_quick\_deploy`.

The only authorization phrase for Phase B is:

APPROVE ARM QUICK DEPLOY: <CI_JOB_NAME> | <BUILD_NUMBER> | <PROJECT_NAME> | <BUILD_TITLE>

Phase B - execute after exact approval

The operator supplied the exact quick-deploy authorization phrase from Phase A.

1. Verify all four identifiers exactly match the approved values.
2. Call `arm\_quick\_deploy` once.
3. Call `arm\_poll\_job\_status` for the exact job and build number.
4. Report HTTP status, initiation message, current build/quick-deploy status, rollback validation flag, and every MCP tool called.
5. Do not retry the mutation automatically. Do not roll back or abort.

nCino mutation

Use the same two-phase pattern with `arm_ncino_trigger_build`. The approval record must include exact `jobName`, `title`, `deploy`, `commitFeature`, and `destination org`. Pass `deploy` and `commitFeature` as JSON booleans.

Source

- ARM mutation tools and behavior
- ARM guided prompts